Structures in C

1. Introduction

In C programming, a **structure** is a user-defined data type that allows us to **group variables of different data types under one name**.

Unlike arrays, which store elements of the **same data type**, structures can store **different types of data**.

Example Situation

Suppose we want to store information about a **student**:

- Student ID
- Name
- Age
- Marks

Using separate variables becomes difficult to manage.

Example:

```
int id;

char name[20];

int age;

float marks;
```

Instead, we can group them into **one structure**.

2. Defining a Structure

A structure is defined using the **struct keyword**.

```
struct structure_name
{
   data_type member1;
   data_type member2;
   data_type member3;
};
```

Example: Student Structure

```
struct Student
{
   int id;
   char name[30];
   int age;
   float marks;
};
```

This structure contains **four members**.

3. Declaring Structure Variables

After defining a structure, we can create **structure variables**.

Syntax

```
struct structure_name variable_name;
```

Example:

```
struct Student s1;
```

Now s1 can store:

- id
- name
- age
- marks

4. Accessing Structure Members

Structure members are accessed using the **dot operator (.)**.

Syntax

```
structure_variable.member
```

Example:

```
s1.id = 101;

s1.age = 20;
```

Example Program

```
#include <stdio.h>

struct Student
{
   int id;
   char name[30];
   float marks;
};

int main()
{
   struct Student s1;

   s1.id = 1;
   s1.marks = 85.5;

   printf("Student ID: %d\n", s1.id);
   printf("Marks: %.2f", s1.marks);

   return 0;
}
```

5. Reading Structure Data from User

Example:

```
#include <stdio.h>

struct Student
{
   int id;
   char name[30];
   float marks;
};

int main()
{
   struct Student s1;

   printf("Enter ID: ");
   scanf("%d", &s1.id);

   printf("Enter Name: ");
   scanf("%s", s1.name);

   printf("Enter Marks: ");
   scanf("%f", &s1.marks);

   printf("\nStudent Information\n");
   printf("ID: %d\n", s1.id);
   printf("Name: %s\n", s1.name);
   printf("Marks: %.2f\n", s1.marks);

   return 0;
}
```

6. Array of Structures

We can create **multiple records using an array of structures**.

Example: storing information for many students.

Syntax

struct Student students[10];

This creates an array that can store **10 student records**.

Example Program

```
#include <stdio.h>

struct Student
{
   int id;
   float marks;
};

int main()
{
   struct Student s[3];

   for(int i = 0; i < 3; i++)
   {
      printf("Enter ID: ");
      scanf("%d", &s[i].id);

      printf("Enter Marks: ");
      scanf("%f", &s[i].marks);
   }
   printf("\nStudent Records\n");

   for(int i = 0; i < 3; i++)
   {
      printf("ID: %d Marks: %.2f\n", s[i].id, s[i].marks);
   }
   return 0;
}
```

7. Initializing Structures

Structures can also be initialized during declaration.

Example:

```
struct Student s1 = {1, "John", 85.5};
```

If structure definition is:

```
struct Student
{
   int id;
   char name[30];
   float marks;
};
```

8. Structures with Functions

Structures can be passed to functions.

Example:

```
void display(struct Student s)
{
   printf("ID: %d\n", s.id);
   printf("Marks: %.2f\n", s.marks);
}
```

9. Nested Structures

A structure can contain **another structure**.

Example:

```
struct Date
{
   int day;
   int month;
   int year;
};

struct Student
{
   int id;
   struct Date dob;
};
```

Example access:

student1.dob.day

Difference Between Array and Structure

Feature	**Array**	**Structure**
Data type	Same	Different
Example	List of numbers	Student record
Access	index	dot operator